

Tools for Measuring and Monitoring the Energy Efficiency of Software Systems: A Rapid Review

Bjorn Pijnacker Jesper van der Zwaag Julian Pasveer

February 11, 2023

Abstract

The recent growth of the information and communication technology (ICT) industry has had a significant impact on the environment. This trend is concerning for both the sustainability of the ICT industry and the global environment. One way to address this issue is to focus on improving the power efficiency of software. For this purpose, a software developer requires tools to investigate their software's power usage.

This paper reviews the literature on available tools for measuring the power efficiency of software and discusses the limitations of these tools. We find that tools can be categorized by target, granularity, and hardware requirements. One limitation of these tools is the need for specialized hardware for some measurements. Another limitation is the accuracy of tools that estimate power usage without specialized hardware.

1 Introduction

The proliferation of computing devices in recent years has led to a significant increase in the overall energy consumption of the information and communication technology (ICT) sector [1]. This trend has important implications for both the sustainability of the ICT industry and the global environment, as the production and consumption of electricity can have negative impacts on air and water quality, as well as contribute to climate change. Moreover, mobile devices like smartphones have limited battery capacity, and energy-inefficiency in apps has been shown to irritate users [2].

The world has recently seen a huge increase in the price of energy [3]. In this context, it is important for individuals and companies who develop their own software applications to ensure that these applications are designed to be as energy-efficient as possible, thereby being as cheap to use as is attainable.

Software engineers have a responsibility to ensure that software is designed and implemented in a way that minimizes its energy consumption. In this review, we will provide an overview and categorization of the tools currently available for measuring the energy consumption of software.

In the rest of this paper, we start off in Section 2 by giving background information in which we moreover describe our motivation and goals for this review. Afterward, we provide our research questions and lay out our research method in Section 3. In Section 4 and Section 6 we engage in our results and conclusion, after which we discuss and investigate future work in Section 5 and Section 7 respectively. Supplemental data, e.g. the data extraction results, can be found in Appendix A.

2 Background

The rapid growth of the information and communication technology (ICT) sector in recent years has resulted in a significant increase in its environmental impact [4]. This is primarily due to the increasing electricity consumption of computing infrastructure, which is

expected to reach several thousand TWh by 2030, representing a significant increase compared to current levels [1]. This trend has important implications for the sustainability of the ICT sector and highlights the need for effective strategies to reduce the environmental impact of technology.

2.1 Motivation and Goals

Given the potential consequences of the growing environmental impact of the ICT sector, it is crucial to develop and implement effective strategies to reduce its negative effects. This may involve a combination of technological solutions, such as developing more energy-efficient computing systems. By addressing the environmental impact of technology, we can ensure the long-term sustainability of the ICT sector and reduce its negative impact on the planet.

A system consists of two main parts: the hardware and the software. The hardware uses the power, but it is the software that determines how the hardware consumes power. It has become clear that it is essential to write power-efficient software. In order to do so, you have to be able to measure the power usage of a (piece of) software. It is, however, unclear which tools are available for this purpose. Measuring how much power a hardware component consumes is quite trivial. Simply measure the power going through the main wires. Measuring how much power a certain piece of software consumes, not so much. There are no ‘main wires’ running to an application, and it is seldom the only piece of software running on a specific piece of hardware.

It is, however, not impossible. But due to the challenges that measuring software power consumption presents, there is a wide variety of proposed solutions.

To lessen the environmental impact caused by technology it is important to write power-efficient software as a software engineer. The objective of this study is to give an overview of available tools, with the sub-goal of classifying these tools into different categories. Moreover, we wish to investigate what insight these tools provide to the user, and what these tools can and cannot do.

3 Research Method

In this section, we will outline our rapid review that will fulfill the goals as outlined in Section 2.1.

3.1 Research Questions

Our goals as outlined in Section 2.1 need to be transformed into formal research questions that we can answer. We pose the following questions and sub-questions:

RQ1 What energy efficiency analysis tools can be used to measure the power efficiency of software?

RQ1.1 How can said analysis tools be categorized?

RQ1.2 What insight do these tools provide?

RQ1.3 What is the target¹ of these tools?

RQ2 What are the limitations of the existing tools?

3.2 Related Work

In this section, we will discuss reviews with similar research questions to ours. We will also look at some papers in the same field as ours, which ask different yet related, questions.

¹We take the target to mean the set of software that the tool can analyze. For example, Android applications, Java systems, or cloud applications.

The field of Green IT has been quickly expanding in the last decade. In fig. 3.1 we show the number of papers available in IEEE Xplore for the query “Green IT” against the year they were published. Clearly, the field has started expanding hugely since approximately 2010.

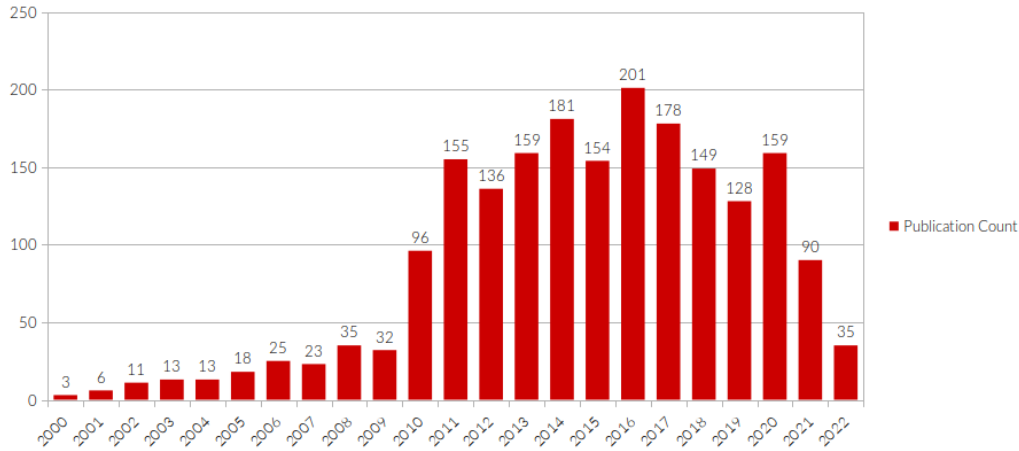


Figure 3.1: Publications matching the query “Green IT” on IEEE Xplore

Related work exists in a few main categories. Firstly, we have studies that consider tools and methods to analyze systems and give meaningful insight into their energy usage. This also includes studies that research specific metrics that can be used to estimate energy usage. The results of these studies are then used again by the respective tooling. Secondly, we have papers that analyze specific pieces of software or specific algorithms. These papers aim to give a software developer a basis for choosing any among the tested subjects to write energy-efficient software.

To estimate software energy usage instead of measuring it, software metrics are required. Previous literature reviews have focused on software sustainability metrics more than the availability and features of actual tools. For example, [5] asks the research question “Are there software sustainability measures and indicators proposed in the literature?” and concludes that 61 measures exist for measuring performance efficiency, maintainability, portability, usability, and reliability in a software system. Performance efficiency, which is of main interest to us, is mainly measured by metrics related to resource utilization. Resource utilization has been shown to be a fair measure of power usage [6].

The focus in the literature has also partly been on big consumers of energy, such as data centers [7]. In these papers, the focus is often on automatically providing energy efficiency by, for example, intelligent scheduling algorithms, more so than actually optimizing the software systems itself.

The last category is papers that focus on measuring a suite of algorithms for some task. For example, [8] finds that JPEG is more energy-efficient than PNG when encoding and decoding, and [9] does a similar comparison for common sorting algorithms.

While tools to measure the energy efficiency of software systems are proposed in papers, to our knowledge there exists no study that looks at multiple of these tools, categorizes them, and provides a good overview for a developer in need of a measurement tool. As such, we will continue by providing such a categorization and overview.

3.3 Search Method

To find papers to perform our rapid review we first define our scope and search query for automatic search in Section 3.4. We then define how we filter studies in Section 3.5, how we assess the quality of found studies in Section 3.6, and how we will perform data extraction and synthesis in Section 3.7 and 3.8 respectively. Furthermore, we will use snowballing. Lastly, we discuss threats to validity in Section 3.9.

3.4 Scoping and Search Query

To find papers to answer the main research questions as posed in Section 3.1 we do an automatic search on IEEE Xplore. This gives us the best overview of multiple publishers and conferences. We propose the following query:

```
("All Metadata":green)
  AND ("All Metadata":analysis
      OR "All Metadata":measure
      OR "All Metadata":profiler)
  AND ("All Metadata":energy efficiency)
  AND ("All Metadata":software)
```

We arrived at this search query on a trial-and-error basis. We reduce our search by only considering papers limited to the "Computer Science" subject area in IEEE Xplore. We find that no further scoping is necessary as we have 240 results, and most results are—at first sight—relevant.

3.5 Study Selection

Using the results of section 3.4 we perform a study selection in multiple rounds. These rounds are described below:

- 1– For the first round of study selection, we filter the set of papers by title. Every paper in the set is reviewed by each of us, and disagreements are discussed. We end up with a smaller set of papers with which we continue to the second round (39 results with the search query, 24 results with snowballing).
- 2– In the second round of study selection, the papers are filtered by abstract. Again, each of us reviews every paper, and disagreements are discussed (23 papers with the search query, 12 papers with snowballing).
- 3– The third round of study selection consists of reading the papers in full. We combine this with the quality assessment and data extraction that we will discuss later. We again include only papers relevant to our research (13 papers with the search query, 8 papers with snowballing). The exact inclusion/exclusion criteria are discussed in section 3.5.1.

In Figure 3.2 we outline the results of the study selection. This figure also shows snowballing. After the initial selection, not enough papers were selected to meet our goals. As such, for every paper, we compiled a list of all their references and performed round 1–3 again. Results from both were then combined to obtain the final list of papers for analysis.

3.5.1 Inclusion/exclusion criteria

To ensure that our literature search is reproducible, in each round we use some inclusion and exclusion criteria to help us guide whether to in- or exclude a paper. We include papers that:

- discuss or propose a tool to measure software energy use.

We exclude papers that:

- analyze a specific software using a tool;
- discuss metrics as opposed to tooling;
- discuss methodology for writing energy efficient software;
- discusses a tool for software that is no longer relevant in the industry (e.g. Adobe Flash [10], Internet Explorer [11]).

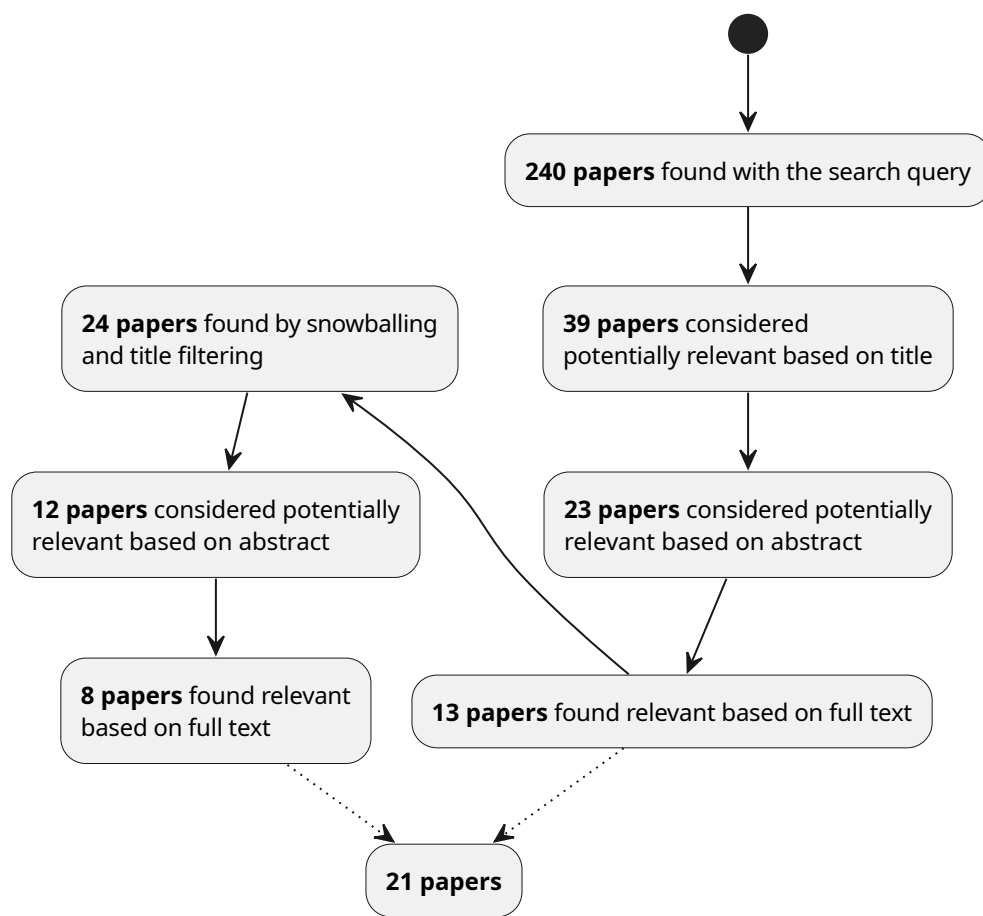


Figure 3.2: Results of study selection rounds

3.6 Quality Assessment

For purposes of quality assessment, we ask six questions for each paper, named below. Each question is answered with either “yes”, “to some extent”, or “no”, resulting in 1, 0.5, or 0 points respectively. For each paper, the points are summed which gives us the final quality score for each paper. However, we do not filter papers based on quality scores, since we do not have very many papers, to begin with.

- 1- Is there a rationale provided for why the study was undertaken?
- 2- Is there an adequate description of the context (industry, laboratory)?
- 3- Is there a justification and description of the research design?
- 4- Is there a clear statement of findings, including data supporting findings?
- 5- Did the researcher(s) critically examine their own role and potential bias?
- 6- Are limitations and credibility of the study discussed explicitly?

3.7 Data Extraction

Our data extraction process will use the form as outlined in Table 3.1. We omit the usual field “type of study” as every study from our search is a primary study.

We include such a form since data extraction is performed by multiple authors. Using this form, we can be consistent in our data extraction and we know what information we are looking for. The completed data extraction is shown in Appendix A.

#	Field	Research question
F1	Author(s)	–
F2	Year	–
F3	Title	–
F4	Venue	–
F5	Keywords	–
F6	Abstract	–
F7	Tools proposed	RQ1
F8	Insight provided by tools	RQ1.2
F9	Tool’s granularity	RQ1.2
F10	Tool’s target	RQ1.3
F11	Limitations	RQ2

Table 3.1: Data extraction form

3.8 Data Synthesis

In this section, we will go over our strategy for data synthesis. The actual results are presented in Section 4.

As seen in Section 3.7, each paper in consideration is given a field on what insight it provides, what granularity the tool provides, and what the target of this tool is. Based on this, we can categorize the tools in question into groups. These groups will be based on what use it has for developers, which is determined by their insight and granularity. Another dimension is determined by the target of the tool, that is, what can it be used on?

We lastly discuss the limitations that each tool has, and see if these too can be generalized for some category.

3.9 Threats to Validity

In this section, we address several potential threats to the validity of our research, including scope and generalizability limitations, as well as challenges related to quality assessment and study selection.

Firstly, the scope of our search was limited due to the nature of the rapid review. We looked at papers from a single source (IEEEExplore) which returned few results. We expanded our set of papers using one round of backward snowballing. This combination means that we may miss some papers, especially more recent ones due to the backward snowballing. No mitigation for this was implemented, although future research could continue with a broader scope.

A second threat is that of our quality assessment. We found that the questions asked were not always fully representative of the type of paper, so scores could be slightly misleading. More so, we did not filter papers based on their quality score due to the limited number of papers found. In more extensive research, these papers should have been excluded from the selection. Another threat that is related to this is the validity of primary searches. We found that of the 21 papers, only 4 discussed the author’s bias, and only 10 discussed limitations at all. This was, however, mitigated partly by using quality venues. Papers of too little validity would not have been included in our search efforts anyway.

A third threat is the process of study selection. In the first stages, we were careful to have every author of this paper review every paper. In later stages, this overlap decreased, and so data extraction and study in- and exclusion may not have been fully accurate. Part of the mitigation implemented for this threat is to have authors who did less data extraction work on more of the data synthesis, thereby having them also review the paper and extraction results.

A fourth threat originates from the papers that we review. Of these papers, a considerable number named that ‘testing is preliminary’ or that ‘we only tested on one computer/program’. Therefore, the results that we review might not be fully representative of the truth. There is no way for us to mitigate this.

Finally, the generalizability of this rapid review is also a potential threat. The findings may not be applicable to other contexts due to the limited scope and selection of papers. Due to the wide variety of tools, with sometimes specific use cases, it is not possible to find around 20 tools that will suit all possible use cases. Future research should aim to address this threat by expanding the scope to enhance the generalizability of the results.

4 Results

In this section, we will outline the results we found during the analysis of the papers. The protocol for obtaining the results is explained in Section 3.

In our analysis, we have found three main factors to use for the categorization of tools: the tool target, the tool granularity, whether the tool does estimation or actual measurement, and the related factor of whether the tool requires hardware or is software only. We will proceed one by one.

4.1 Categorization

We show the categorization of targets in Table 4.1. In general, the tools found in our analysis have one of four main targets: Java applications, general-purpose applications/computing, cloud/data center applications, and Android/smartphone applications. We also found some tools on IoT and on measuring software tests. By far most of the tools target general software systems and Android/smartphone applications. The latter of course makes sense, as many of these papers name that these systems have limited batteries.

We continue with the categorization of granularity in Table 4.2. We see that most tools either target the application level or the class/component level. Some tools also target

Target	Tool	
Java	GreenSoM	[13]
	<i>no name</i>	[14]
General	SKD4ED (CLANG-compiled software)	[15]
	Energy Toolbox (aimed at IoT)	[16]
	FEETINGS/EET	[12]
	<i>no name</i>	[17]
	<i>no name</i> (Linux)	[18]
	SPELL	[19]
	<i>no name</i>	[20]
	<i>no name</i>	[21]
	SEFLab	[22]
Android/Smartphones	eCalc	[23]
	GreenOracle	[24]
	Orka	[25]
	eProf	[26]
Data centers/Cloud	eTune (data-intensive software)	[27]
	<i>no name</i>	[28]
Other	Green-JEXJ (concolic testing systems)	[29]
	PowerScope (modified kernel system)	[30]

Table 4.1: Tool categorized by target

the thread/process level and the function level. Some tools are even configurable to the extent that they fit into multiple categories.

The last categorization we make is on technique. We filter both on hardware or software-only analysis, and on measurement or estimation. We show this categorization in Table 4.3.

4.2 Limitations

Our analysis of the tools revealed three main limitations: the requirement for specialized hardware, the possibility of not measuring every component in a system, and the potential for inaccurate estimations. These limitations apply to the different classes of tools in varying ways: the need for specialized hardware only applied to tools utilizing measurement hardware, while the potential for inaccurate measurements only applied to estimation tools, as they do not directly measure energy but rather use metrics or models for estimation. The possibility of not measuring every component that uses energy in a system was observed across all tools. No other limitations were identified that could be considered generalizable to the entire range of tools discussed.

Granularity	Tool	
System-level	<i>no name</i>	[28]
Application-level	SKD3ED Energy Toolbox	[15]
	<i>no name</i>	[31]
	eCalc	[23]
	<i>no name</i>	[17]
	eTune	[27]
	Green-JEXT	[29]
	GreenOracle	[24]
	<i>no name</i>	[21]
Thread/process-level	<i>no name</i>	[18]
	PowerScope	[30]
	eProf	[26]
Component/class-level	GreenSoM	[13]
	<i>no name</i>	[14]
	FEETINGS/EET (hardware-component level)	[12]
	eCalc	[23]
	eTune	[27]
	SPELL	[19]
	<i>no name</i>	[20]
	SEFLab (hardware-component level)	[22]
Procedure/function/method-level	Energy Toolbox (can also do per-loop analysis)	[16]
	Orka	[25]
	PowerScope	[30]
	eProf	[26]

Table 4.2: Tool categorized by granularity. Note that some tools appear more than once

Technique		Tool	
Hardware	Measurement	GreenSoM	[13]
		FEETINGS/EET	[12]
		<i>no name</i>	[20]
		<i>no name</i>	[21]
		<i>no name</i>	[28]
		SEFLab	[22]
	Estimation	<i>no name</i> (power meter required for initial setup)	[18]
Software Only	Measurement	Orka (measures emulator)	[25]
		<i>no name</i>	[31]
		PowerScope	[30]
		Green-JEXJ	[29]
	Estimation	SKD4ED	[15]
		<i>no name</i>	[14]
		Energy Toolbox	[16]
		eCalc	[23]
		<i>no name</i>	[17]
		eTune	[27]
		GreenOracle	[24]
		SPELL	[19]
		ePROF	[26]

Table 4.3: Tool categorized by technique. Note that some tools appear more than once

5 Discussion

During our analysis, we categorized the tools in three main ways: by target, by granularity, and by technique. We noticed that there is not necessarily a specific tool for generally used software. A few can, but they lack the benefits the other papers introduced. Most papers only discussed tools that were only applicable to (very) specific software or hardware. Although they may introduce some additional benefits, such as tracking down high-power-consuming parts of the software to the source code.

Some papers (see Table 4.3) introduce particular hardware that one could use as a tool for measuring how energy-efficient a (piece of) code is. This can be inconvenient for certain software developers. Furthermore, several papers discuss estimation techniques. These can be somewhat imprecise and hence would not be the best way to measure energy efficiency if absolute precision is required.

6 Conclusion

In this paper, we set out to answer the question of what energy efficiency analysis tools can be used to measure the power efficiency of software. We performed a rapid review in which 21 papers were analyzed. Using this, we found that tools for energy-efficiency analysis of software can be categorized in three main ways: by target, by granularity, and by technique.

We found that the main limitations to tools are that they may use specialized hardware, which might not be available nor practical to every software developer, and that tools which use estimation techniques may not be completely accurate, although their estimate is often precise enough to give a good indication of power usage and possible improvements to the software.

7 Future Research

Based on our analysis, it is clear that there is a need for further research in the area of energy efficiency analysis tools for software. In particular, there is a need for tools that are applicable to a wider range of software and that do not require specialized hardware. We see a trade-off between granularity and the scope of a target, which is likely unavoidable, but modern tools for these wide ranges of software can prove useful. Additionally, there is a need for more accurate estimation techniques that can be used to measure the power efficiency of software. Many of the papers considered here that focus on estimation provide an error of approximately 10%, often with limited testing.

Future research could focus on developing new tools that address these limitations. For example, researchers could explore the use of machine learning algorithms to create more accurate estimation techniques or develop tools that can be used with commonly available hardware. Additionally, the research could be conducted to investigate the feasibility of using these tools in different software development environments, including those with limited resources.

Ultimately, the goal of this research would be to create a set of tools that are widely applicable, accurate, available, and easy to use, so that software developers can easily measure and improve the power efficiency of their software. This would have the potential to significantly reduce the energy consumption of software systems, contributing to a more sustainable future of IT.

8 References

8.1 References used in paper

- [1] A. Andrae, "Prediction Studies of Electricity Use of Global Computing in 2030," vol. 8, pp. 27–33, Mar. 31, 2019.
- [2] C. Wilke, S. Richly, S. Götz, C. Piechnick, and U. Assmann, "Energy Consumption and Efficiency in Mobile Applications: A User Feedback Study," Aug. 21, 2013. DOI: 10.1109/GreenCom-iThings-CPSCoM.2013.45.
- [3] "Energy price rise since 2021." (), [Online]. Available: <https://www.consilium.europa.eu/en/infographics/energy-prices-2021/> (visited on 12/14/2022).
- [4] J. Mancebo, F. García, and C. Calero, "A process for analysing the energy efficiency of software," *Information and Software Technology*, vol. 134, p. 106560, Jun. 1, 2021, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2021.106560. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584921000446> (visited on 11/20/2022).
- [5] C. Calero, M. F. Bertoa, and M. Á. Moraga, "A systematic literature review for software sustainability measures," in *2013 2nd International Workshop on Green and Sustainable Software (GREENS)*, May 2013, pp. 46–53. DOI: 10.1109/GREENS.2013.6606421.
- [6] E. Jagroep, V. D. Jan Martijn Werf, J. Broekman, L. Blom, V. Rob Vliet, and S. Brinkkemper, "A Resource Utilization Score for Software Energy Consumption," presented at the ICT for Sustainability 2016, Atlantis Press, Aug. 2016, pp. 19–28, ISBN: 978-94-6252-224-4. DOI: 10.2991/ict4s-16.2016.3. [Online]. Available: <https://www.atlantis-press.com/proceedings/ict4s-16/25860363> (visited on 12/04/2022).
- [7] F. Khan, H. Anwar, D. Pfahl, and S. Srirama, "Software Techniques for Making Cloud Data Centers Energy-efficient: A Systematic Mapping Study," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, Aug. 2020, pp. 479–486. DOI: 10.1109/SEAA51224.2020.00081.
- [8] M. Rashid, L. Ardito, and M. Torchiano, "Energy Consumption Analysis of Image Encoding and Decoding Algorithms," in *2015 IEEE/ACM 4th International Workshop on Green and Sustainable Software*, May 2015, pp. 15–21. DOI: 10.1109/GREENS.2015.10.
- [9] N. Schmitt, S. Kamthania, N. Rawtani, L. Mendoza, K.-D. Lange, and S. Kounev, "Energy-Efficiency Comparison of Common Sorting Algorithms," in *2021 29th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Nov. 2021, pp. 1–8. DOI: 10.1109/MASCOTS53633.2021.9614299.
- [10] M. E. Blog. "Update on Adobe Flash Player End of Support," Microsoft Edge Blog. (Sep. 4, 2020), [Online]. Available: <https://blogs.windows.com/msedgedev/2020/09/04/update-adobe-flash-end-support/> (visited on 12/10/2022).
- [11] S. Lyndersay. "Internet Explorer 11 has retired and is officially out of support—what you need to know," Internet Explorer 11 has retired and is officially out of support—what you need to know. (Jun. 15, 2022), [Online]. Available: <https://blogs.windows.com/windowsexperience/2022/06/15/internet-explorer-11-has-retired-and-is-officially-out-of-support-what-you-need-to-know/> (visited on 12/11/2022).
- [12] J. Mancebo, H. O. Arriaga, F. García, M. Á. Moraga, I. G.-R. de Guzmán, and C. Calero, "EET: A device to support the measurement of software consumption," in *Proceedings of the 6th International Workshop on Green and Sustainable Software*, ser. GREENS '18, New York, NY, USA: Association for Computing Machinery, May 27, 2018, pp. 16–22, ISBN: 978-1-4503-5732-6. DOI: 10.1145/3194078.3194081. [Online]. Available: <http://doi.org/10.1145/3194078.3194081> (visited on 11/20/2022).

8.2 References found during review

- [13] V. Cordero, I. G.-R. de Guzmán, and M. Piattini, "A First Approach on Legacy System Energy Consumption Measurement," in *2015 IEEE 10th International Conference on Global Software Engineering Workshops*, Jul. 2015, pp. 35–43. DOI: 10.1109/ICGSEW.2015.15.
- [14] C. Seo, S. Malek, and N. Medvidovic, "An energy consumption framework for distributed java-based systems," in *Proceedings of the Twenty-Second IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '07, New York, NY, USA: Association for Computing Machinery, Nov. 5, 2007, pp. 421–424, ISBN: 978-1-59593-882-4. DOI: 10.1145/1321631.1321699. [Online]. Available: <http://doi.org/10.1145/1321631.1321699> (visited on 12/07/2022).

- [15] C. Marantos, K. Salapas, L. Papadopoulos, and D. Soudris, "A Flexible Tool for Estimating Applications Performance and Energy Consumption Through Static Analysis," *SN Computer Science*, vol. 2, no. 1, p. 21, Jan. 7, 2021, ISSN: 2661-8907. DOI: 10.1007/s42979-020-00405-7. [Online]. Available: <https://doi.org/10.1007/s42979-020-00405-7> (visited on 12/07/2022).
- [16] C. Marantos, L. Papadopoulos, C. P. Lamprakos, K. Salapas, and D. Soudris, "Bringing Energy Efficiency closer to Application Developers: An Extensible Software Analysis Framework," *IEEE Transactions on Sustainable Computing*, pp. 1–14, 2022, ISSN: 2377-3782. DOI: 10.1109/TSUSC.2022.3222409.
- [12] J. Mancebo, H. O. Arriaga, F. García, M. Á. Moraga, I. G.-R. de Guzmán, and C. Calero, "EET: A device to support the measurement of software consumption," in *Proceedings of the 6th International Workshop on Green and Sustainable Software*, ser. GREENS '18, New York, NY, USA: Association for Computing Machinery, May 27, 2018, pp. 16–22, ISBN: 978-1-4503-5732-6. DOI: 10.1145/3194078.3194081. [Online]. Available: <http://doi.org/10.1145/3194078.3194081> (visited on 11/20/2022).
- [17] C. Fu, D. Qian, and Z. Luan, "Estimating Software Energy Consumption with Machine Learning Approach by Software Performance Feature," in *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, Jul. 2018, pp. 490–496. DOI: 10.1109/Cybermatics_2018.2018.00106.
- [18] V. K. Singh, K. Dutta, and D. VanderMeer, "Estimating the Energy Consumption of Executing Software Processes," in *2013 IEEE International Conference on Green Computing and Communications and IEEE Internet of Things and IEEE Cyber, Physical and Social Computing*, Aug. 2013, pp. 94–101. DOI: 10.1109/GreenCom-iThings-CPSCom.2013.40.
- [19] R. Pereira, T. Carção, M. Couto, J. Cunha, J. P. Fernandes, and J. Saraiva, "Helping Programmers Improve the Energy Efficiency of Source Code," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, May 2017, pp. 238–240. DOI: 10.1109/ICSE-C.2017.80.
- [20] T. Johann, M. Dick, S. Naumann, and E. Kern, "How to measure energy-efficiency of software: Metrics and measurement results," in *2012 First International Workshop on Green and Sustainable Software (GREENS)*, Jun. 2012, pp. 51–54. DOI: 10.1109/GREENS.2012.6224256.
- [21] A. Hankel, R. van den Hoed, E. Hoekstra, and R. van Rijswijk, "Measuring software energy efficiency: Presenting a methodology and case study on DNS resolvers," in *2016 18th Mediterranean Electrotechnical Conference (MELECON)*, Apr. 2016, pp. 1–6. DOI: 10.1109/MELCON.2016.7495390.
- [22] M. A. Ferreira, E. Hoekstra, B. Merkus, B. Visser, and J. Visser, "Seflab: A lab for measuring software energy footprints," in *2013 2nd International Workshop on Green and Sustainable Software (GREENS)*, May 2013, pp. 30–37. DOI: 10.1109/GREENS.2013.6606419.
- [23] S. Hao, D. Li, W. G. J. Halfond, and R. Govindan, "Estimating Android applications' CPU energy usage via bytecode profiling," in *2012 First International Workshop on Green and Sustainable Software (GREENS)*, Jun. 2012, pp. 1–7. DOI: 10.1109/GREENS.2012.6224263.
- [24] S. A. Chowdhury and A. Hindle, "GreenOracle: Estimating Software Energy Consumption with Energy Measurement Corpora," in *2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR)*, May 2016, pp. 49–60.
- [25] B. Westfield and A. Gopalan, "Orka: A new technique to profile the energy usage of Android applications," in *2016 5th International Conference on Smart Cities and Green ICT Systems (SMARTGREENS)*, Apr. 2016, pp. 1–12.
- [26] A. Pathak, Y. C. Hu, and M. Zhang, "Where is the energy spent inside my app? fine grained energy accounting on smartphones with Eprof," in *Proceedings of the 7th ACM European Conference on Computer Systems*, ser. EuroSys '12, New York, NY, USA: Association for Computing Machinery, Apr. 10, 2012, pp. 29–42, ISBN: 978-1-4503-1223-3. DOI: 10.1145/2168836.2168841. [Online]. Available: <http://doi.org/10.1145/2168836.2168841> (visited on 12/07/2022).
- [27] R. Ge, X. Feng, T. Wirtz, Z. Zong, and Z. Chen, "eTune: A Power Analysis Framework for Data-Intensive Computing," in *2012 41st International Conference on Parallel Processing Workshops*, Sep. 2012, pp. 254–261. DOI: 10.1109/ICPPW.2012.38.
- [28] T. Ilsche, D. Hackenberg, S. Graul, R. Schöne, and J. Schuchart, "Power measurements for compute nodes: Improving sampling rates, granularity and accuracy," in *2015 Sixth International Green and Sustainable Computing Conference (IGSC)*, Dec. 2015, pp. 1–8. DOI: 10.1109/IGCC.2015.7393710.

- [29] S. Godbole, A. Dutta, B. Besra, and D. P. Mohapatra, "Green-JEXJ: A new tool to measure energy consumption of improved concolic testing," in *2015 International Conference on Green Computing and Internet of Things (ICGCIoT)*, Oct. 2015, pp. 36–41. DOI: 10.1109/ICGCIoT.2015.7380424.
- [30] J. Flinn and M. Satyanarayanan, "PowerScope: A tool for profiling the energy usage of mobile applications," in *Proceedings WMCSA'99. Second IEEE Workshop on Mobile Computing Systems and Applications*, Feb. 1999, pp. 2–10. DOI: 10.1109/MCSA.1999.749272.
- [31] F. Chen, J.-G. Schneider, Y. Yang, J. Grundy, and Q. He, "An energy consumption model and analysis tool for Cloud computing environments," in *2012 First International Workshop on Green and Sustainable Software (GREENS)*, Jun. 2012, pp. 45–50. DOI: 10.1109/GREENS.2012.6224255.
- [32] F. Chen, J. Grundy, J.-G. Schneider, Y. Yang, and Q. He, "Automating Performance and Energy Consumption Analysis for Cloud Applications," in *2015 IEEE World Congress on Services*, Jun. 2015, pp. 63–70. DOI: 10.1109/SERVICES.2015.18.

A Data extraction results

In this section, we provide two tables. The first, table A.1, shows the quality assessment results as we outlined in section 3.6. The second table (Table A.2, next page) show the results of the data extraction.

Score	Paper
6.0	[24]
5.5	
5.0	[32, 21, 25, 22]
4.5	[13, 23, 20, 28, 30]
4.0	[15, 14, 16, 17, 18, 27, 29, 26]
3.5	[12]
3.0	
2.5	[31]
2.0	[19]
1.5	
1.0	
0.5	
0.0	

Table A.1: Quality Assessment Results

Table A.2: Data Extraction Results

[13]	Metric	Value
	Tools proposed	GreenSoM
	Insight provided by tool	Runtime energy measurement
	Tool's granularity	Java-class level
	Tool's target	Legacy Java systems
	Limitations	Tool changes energy usage of measured system; Data logger hardware required
[15]	Metric	Value
	Tools proposed	SKD4ED Energy Toolbox
	Insight provided by tool	Compile-time energy-usage estimation
	Tool's granularity	Application-level
	Tool's target	CLANG-compiled software
	Limitations	Testing only performed on single-core workloads
[14]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	Run- and construction-time program energy estimation
	Tool's granularity	Component-level
	Tool's target	Distributed Java-based systems
	Limitations	<i>none named</i>
[31]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	Runtime energy measurement
	Tool's granularity	Task/application-level
	Tool's target	Cloud computing tasks of a single type: data-intensive, computation-intensive, communication-intensive
	Limitations	Mixed-type tasks not tested
[16]	Metric	Value
	Tools proposed	Energy Toolbox
	Insight provided by tool	Energy usage estimation without need for target device
	Tool's granularity	Per-function or per-loop (configurable)
	Tool's target	IoT devices
	Limitations	Accuracy, only CPU and CPU-GPU architectures, the framework is not fully automated
[12]	Metric	Value
	Tools proposed	FEETINGS/EET
	Insight provided by tool	All-component energy usage during application
	Tool's granularity	Hardware-component
	Tool's target	General-purpose PCs/applications
	Limitations	Requires specialized hardware
[23]	Metric	Value
	Tools proposed	eCalc
	Insight provided by tool	Application energy usage estimation
	Tool's granularity	Method or whole program
	Tool's target	Android applications
	Limitations	Used control is not 100% accurate
[17]	Metric	Value

Continued on next page

Table A.2: Data Extraction Results (Continued)

	Tools proposed	<i>no name</i>
	Insight provided by tool	Classification of software energy usage based on performance features
	Tool's granularity	Program
	Tool's target	Communication networks, personal computers, and data centers
	Limitations	Proposed models could be tested on more applications
[18]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	Energy measurement
	Tool's granularity	Process-level
	Tool's target	Linux applications/processes
	Limitations	Power-meter required for setup
[27]	Metric	Value
	Tools proposed	eTune
	Insight provided by tool	Power-profiling estimation
	Tool's granularity	System, node, component
	Tool's target	Data-intensive data centre software
	Limitations	Currently only measures CPU
[29]	Metric	Value
	Tools proposed	Green-JEXJ
	Insight provided by tool	Measuring energy consumption
	Tool's granularity	Total energy usage
	Tool's target	Concolic testing systems
	Limitations	Only tested on 3 sample programs
[24]	Metric	Value
	Tools proposed	GreenOracle
	Insight provided by tool	System power energy prediction
	Tool's granularity	Complete application
	Tool's target	Android applications
	Limitations	Application select for testing was manual; tests were only performed on a single device
[19]	Metric	Value
	Tools proposed	SPELL
	Insight provided by tool	Statistical estimation of energy hotspots in code
	Tool's granularity	Packages, classes, methods, functions, lines of code, etc
	Tool's target	Language independent
	Limitations	Study is only preliminary
[20]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	Measure and monitor software energy consumption during development
	Tool's granularity	Sections of software
	Tool's target	Various
	Limitations	Requires specific hardware
[21]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	Compare software with similar functionality in terms of energy efficiency

Continued on next page

Table A.2: Data Extraction Results (Continued)

	Tool's granularity	System-level
	Tool's target	General purpose applications on systems/servers
	Limitations	Requires measuring target hardware modifications
[25]	Metric	Value
	Tools proposed	Orka
	Insight provided by tool	Software and energy hardware usage measurement on emulator; identify hotspots
	Tool's granularity	Method-level
	Tool's target	Android OS
	Limitations	Results depend on test flow created by developer. There can be bias involved
[28]	Metric	Value
	Tools proposed	<i>no name</i>
	Insight provided by tool	High-detail energy measurements for compute nodes
	Tool's granularity	CPU & Socket power
	Tool's target	Compute nodes
	Limitations	Does not measure other components than CPU
[30]	Metric	Value
	Tools proposed	PowerScope
	Insight provided by tool	Runtime energy usage measurement
	Tool's granularity	Process/procedure-level
	Tool's target	Any software that can run on the required modified kernel
	Limitations	Requires a modified kernel to run tests
[22]	Metric	Value
	Tools proposed	SEFLab
	Insight provided by tool	Relation between power dissipation and computational resources
	Tool's granularity	Hardware-component-level
	Tool's target	Any system that can have the required sensors integrated
	Limitations	Requires specialized hardware
[26]	Metric	Value
	Tools proposed	eProf
	Insight provided by tool	Profiling energy usage on smartphones
	Tool's granularity	System call, routine, per-thread, per-process
	Tool's target	Smartphones
	Limitations	None named